DEPLOYMENT OF ONLINE FOOD ORDERING AND DELIVERY APPLICATION

A DevOps Implementation for a Scalable Food Delivery Platform

---

Domain

Food Delivery & E-Commerce | Cloud Infrastructure & DevOps

Prepared by: Parikshit A

Cloud Provider: Amazon Web Services (AWS)

Region: us-east-1 (N. Virginia)

Date: August 2026

Table of Contents

1. Project Overview 3
2. Architecture & AWS Networking Foundation 4
3. Amazon EKS Cluster Provisioning 5
4. Containerization with Docker 7
5. CI/CD Pipeline (Jenkins) 10
6. Kubernetes Deployment & Verification 13
7. Monitoring & Observability 15
8. Security & Identity Access Management (IAM) 17
9. Source Control 19
10. Summary & Outcomes 20

1. Project Overview

Zomato is a leading online food-delivery and restaurant-discovery platform. To keep pace with growing user demand and to improve the speed and reliability of software releases, the company is migrating a monolithic web application to a containerized, microservices-friendly architecture built on modern DevOps practices.

This document records the end-to-end implementation of that migration: provisioning the AWS network and compute foundation, containerizing the application, building an automated CI/CD pipeline, deploying the workload onto Kubernetes, and standing up real-time monitoring — with supporting evidence captured directly from the AWS Console, Jenkins, GitHub, and Grafana.

1.1 Objectives

- Replace manual, error-prone deployments with an automated CI/CD pipeline.
- Containerize the application using Docker for consistent, portable deployments.
- Host the workload on Amazon Elastic Kubernetes Service (EKS) for scalability and high availability.
- Store and version container images in Amazon Elastic Container Registry (ECR).
- Scan images and source dependencies for vulnerabilities before deployment (Trivy).
- Provide real-time cluster observability using Prometheus and Grafana.
- Enforce least-privilege access between AWS services using IAM roles and policies.

1.2 Technology Stack

Layer	Technology
Cloud Provider	Amazon Web Services (AWS) — Region us-east-1
Networking	Amazon VPC, Public Subnets, Internet Gateway, Route Tables
Compute / Orchestration	Amazon EKS (Kubernetes 1.36), EC2 worker nodes (t3.medium)
Containerization	Docker
Container Registry	Amazon Elastic Container Registry (ECR)
CI / CD	Jenkins (Declarative Pipeline)
Security Scanning	Trivy (filesystem & image scanning)
Monitoring & Observability	Prometheus, Grafana, kube-state-metrics, Alertmanager
Identity & Access	AWS IAM Roles & Managed Policies
Source Control	GitHub
Frontend	React

2. Architecture & AWS Networking Foundation

The foundation of the deployment is a dedicated Amazon VPC that isolates the application's network from other workloads. Two public subnets, spread across two Availability Zones, give the cluster fault tolerance and a direct path to the internet through an Internet Gateway.

2.1 Virtual Private Cloud (VPC)

VPC ID	vpc-07f2d70dacaa1b60e
IPv4 CIDR	10.0.0.0/16
Default VPC	No (custom VPC)
Subnets	2 — PUBLIC-subnet-1 (us-east-1a), Public-Subnet-2 (us-east-1b)
Route Tables	2 — RT-1 (rtb-087158edcdae94f9d)
Network Connections	1 — Internet Gateway (INTERNET)

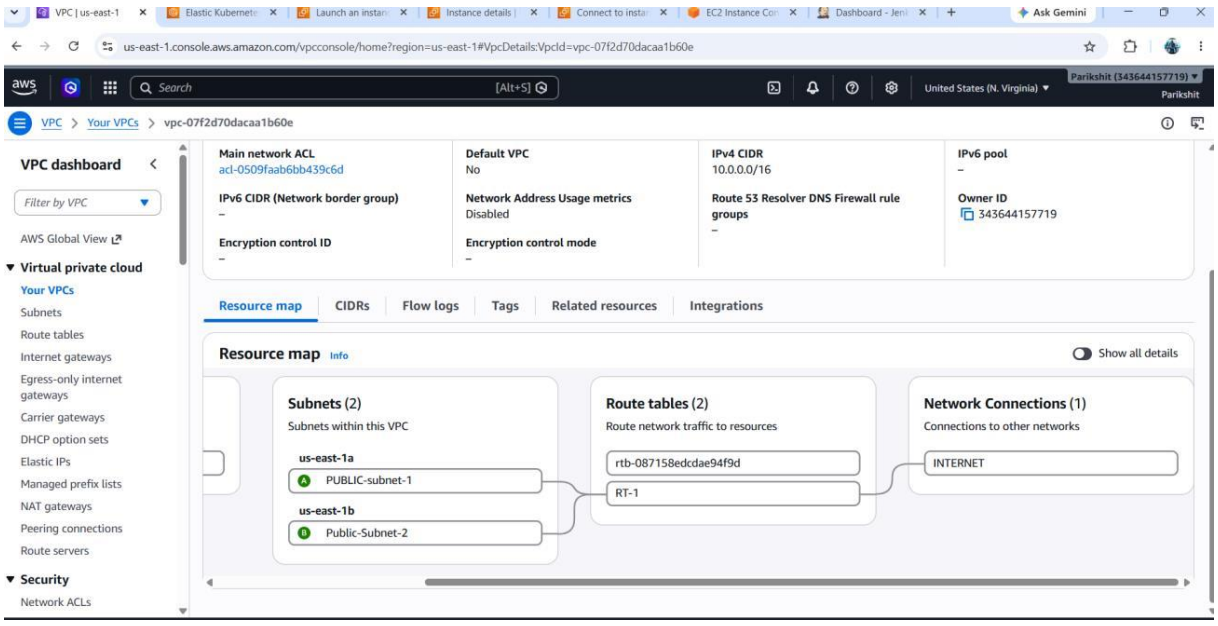


Figure 2.1 — AWS VPC resource map showing subnets, route tables, and internet connectivity.

Spreading the two public subnets across separate Availability Zones (us-east-1a and us-east-1b) ensures the EKS worker nodes — and therefore the application — remain available even if one Availability Zone experiences an outage.

3. Amazon EKS Cluster Provisioning

3.1 EKS Cluster — "Zomato-EKS"

An Amazon EKS cluster was provisioned to orchestrate the containerized workload.

Cluster Name	Zomato-EKS
Status	Active
Kubernetes Version	1.36
Provider	EKS (managed control plane)
Support Period	Standard support until August 2, 2027
Cluster Health	0 issues
Upgrade Insights	4 recommendations
Node Health Issues	0
Cluster ARN	arn:aws:eks:us-east-1:<account-id>:cluster/Zomato-EKS

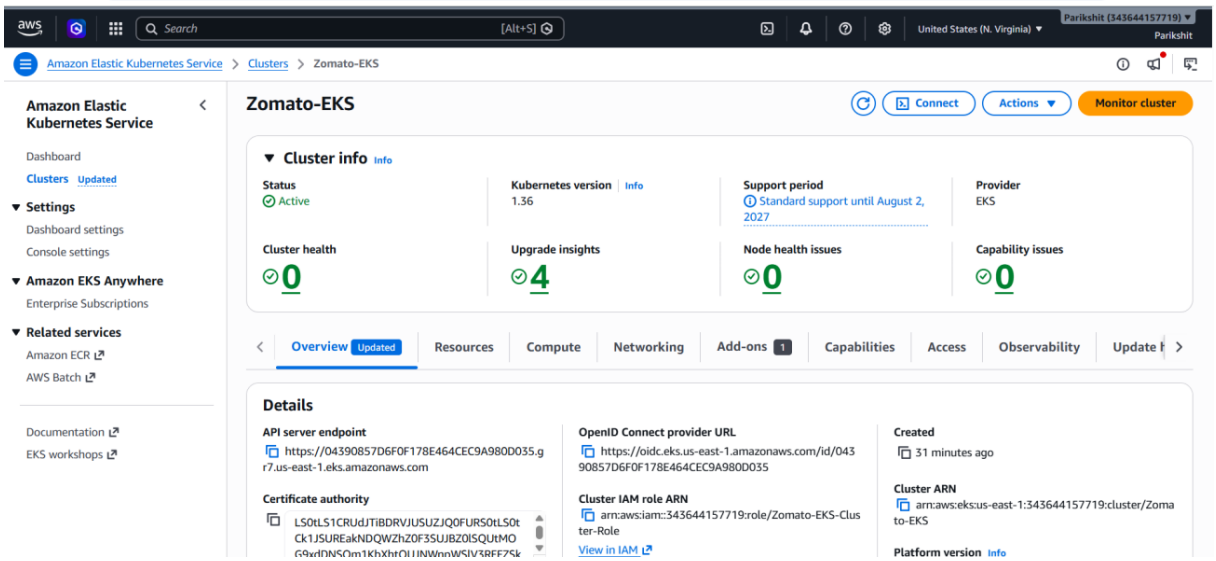


Figure 3.1 — EKS cluster overview showing healthy status and Kubernetes 1.36 control plane.

3.2 EKS Managed Node Group

A managed node group provisions and maintains the EC2 worker nodes that run application and monitoring pods, with autoscaling boundaries so capacity can grow with demand.

Node Group Name	Zomato-Node-group
Kubernetes Version	1.36
AMI Type	Amazon Linux 2023 (x86_64) Standard

Instance Type	t3.medium
Disk Size	20 GiB
Capacity Type	On-Demand
Desired / Min / Max Size	2 / 2 / 3 nodes
Node IAM Role	Zomato-EKS_NODe-Role
Remote Access	Off (no direct SSH to nodes)

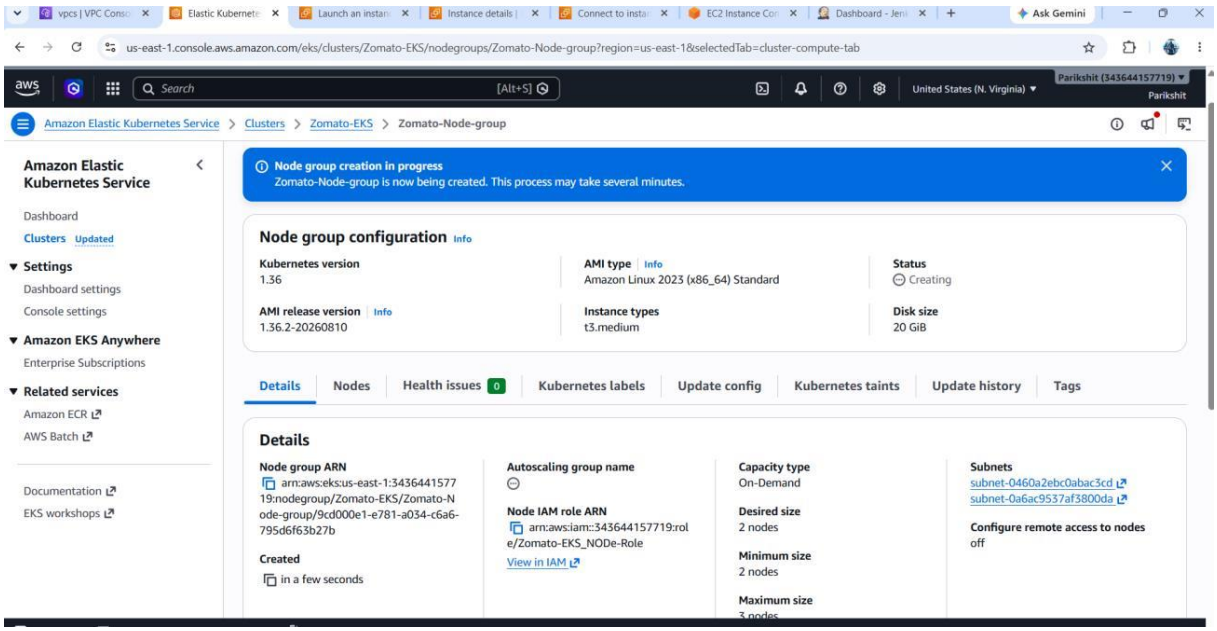


Figure 3.2 — Managed node group configuration for the Zomato-EKS cluster.

4. Containerization with Docker

The React frontend and its supporting services were packaged into a single Docker image so the application runs identically across development, staging, and the EKS cluster.

4.1 Building the Image

The image was built and validated locally on the build EC2 instance before being pushed to the registry.

Image	zomato-app:latest
Image ID	98d24e261562
Image Size	99.7 MB (29.2 MB unique content)
Base Images Used	node:20-alpine (194 MB), nginx:alpine (93.6 MB)
Local Test Run	docker run -d --name zomato-test -p 8081:80 zomato-app:latest

```
Successfully built 98d24e261562
Successfully tagged zomato-app:latest
ubuntu@ip-172-31-14-176:~/Zomato$ docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:alpine         4a73073bd557       93.6MB           27MB
node:20-alpine       fb4cd12c85ee        194MB         48.8MB
zomato-app:latest    98d24e261562       99.7MB         29.2MB
ubuntu@ip-172-31-14-176:~/Zomato$ docker run -d --name zomato-test -p 8081:
7d11285d616b1679cce52e0cc55eeecc3656692cdd5c7c2ed9f12fdcc6bb1f44
```

Figure 4.1 — Successful local Docker build and image list.

4.2 Tagging & Pushing to Amazon ECR

Once verified locally, the image was tagged with the private ECR repository URI and pushed layer-by-layer to Amazon Elastic Container Registry.

ECR Repository URI	<account-id>.dkr.ecr.us-east-1.amazonaws.com/zomato-app
Pushed Tag	latest
Layers Pushed	9 layers
Image Digest	sha256:98d24e261562d88a57e4e27a4c9000b46c256e98ed7a8651ece6fc3573cf4d22
Manifest Size	2265 bytes

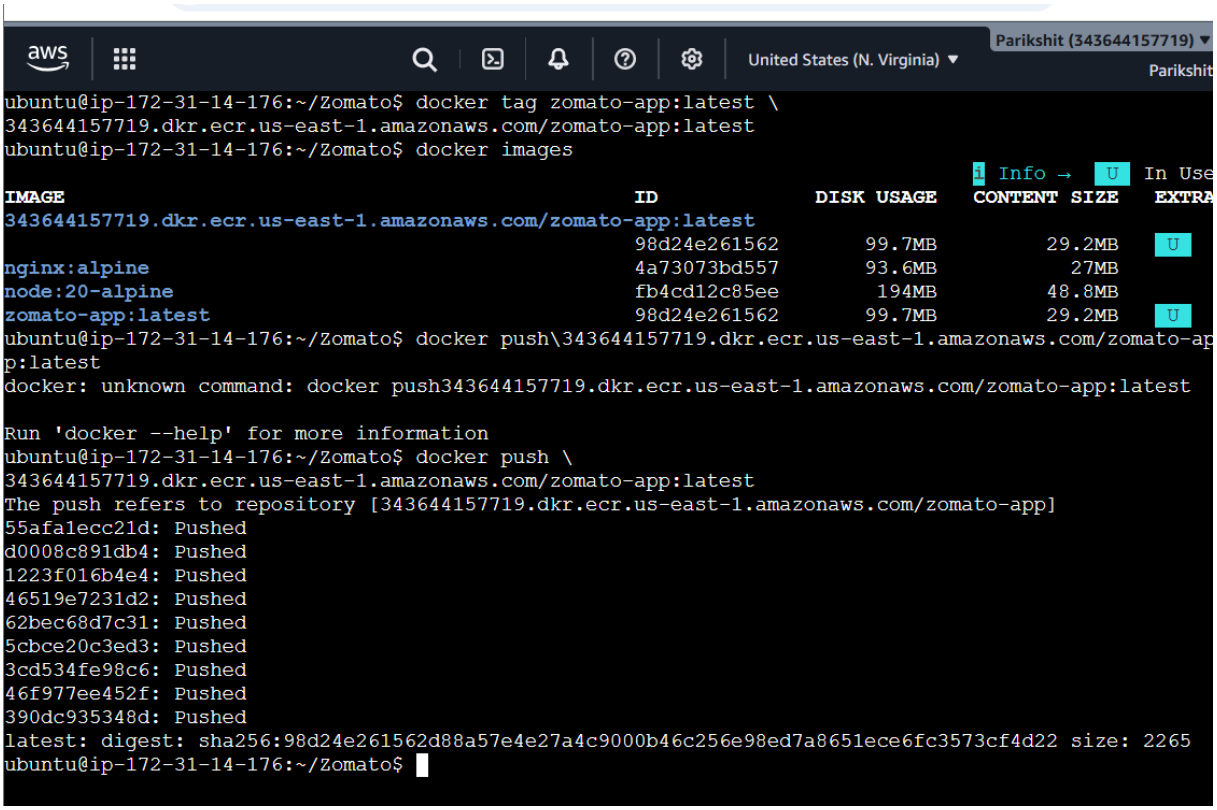


Figure 4.2 — Tagging and pushing the image to the private ECR repository.

4.3 Amazon ECR Repository

The pushed image is versioned and stored in a private ECR repository, ready to be pulled by the EKS cluster.

Repository	zomato-app (private)
Images Stored	1 (tag: latest)
Image Size	29.20 MB
Created	August 15, 2026, 01:29:05 (UTC+05:30)

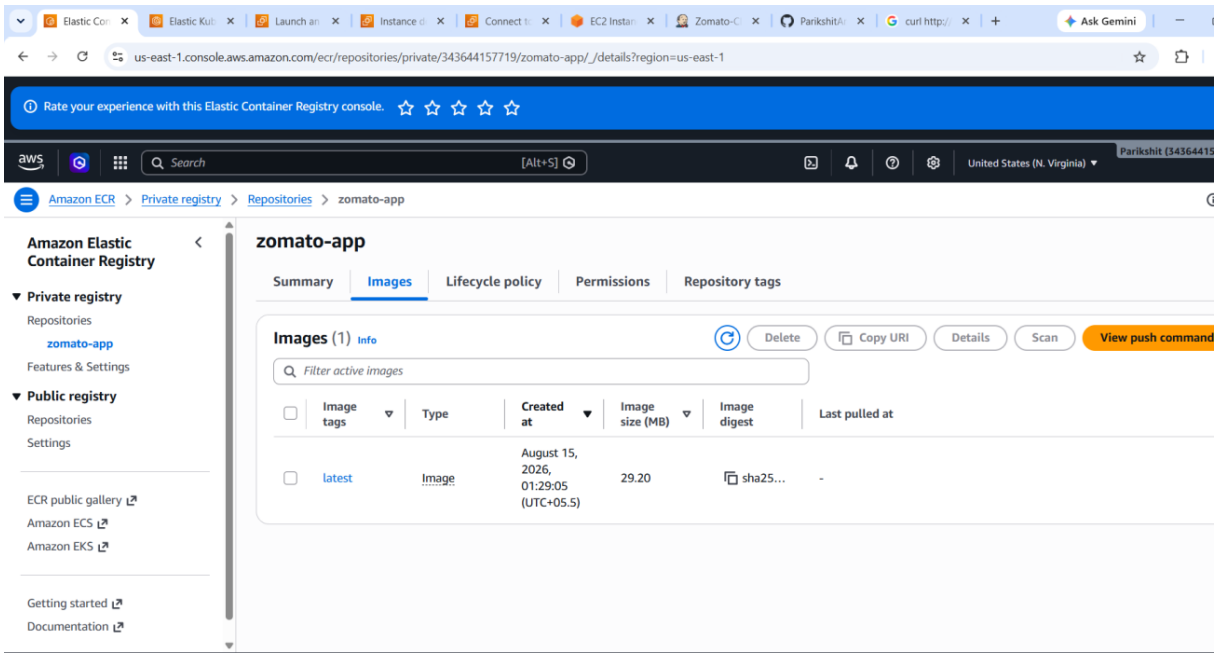


Figure 4.3 — Amazon ECR repository listing the pushed zomato-app image.

5. CI/CD Pipeline (Jenkins)

A declarative Jenkins pipeline, "Zomato-CI_CD", automates the full path from source-code commit to a running deployment on EKS. Each pipeline run executes twelve stages in sequence, giving fast, auditable feedback at every step of the release.

5.1 Pipeline Stages

#	Stage	Purpose
1	Git Checkout	Pull the latest source code from GitHub
2	Install Dependencies	Install Node/React package dependencies
3	Build React App	Compile and bundle the production frontend
4	Trivy Filesystem Scan	Scan source & dependencies for known vulnerabilities
5	Docker Build	Build the container image (zomato-app:latest)
6	Trivy Image Scan	Scan the built image for CVEs before it is shipped
7	Login to ECR	Authenticate Docker with Amazon ECR
8	Push Image to ECR	Push the tagged image to the private repository
9	Connect to EKS	Update kubeconfig / authenticate to the EKS cluster
10	Deploy to EKS	Apply Kubernetes manifests to roll out the new image
11	Verify Deployment	Confirm pods reach a Running / Ready state
12	Declarative: Post Actions	Post-build notifications and cleanup

Table 5.1 — Stages executed on every run of the Zomato-CI_CD pipeline.

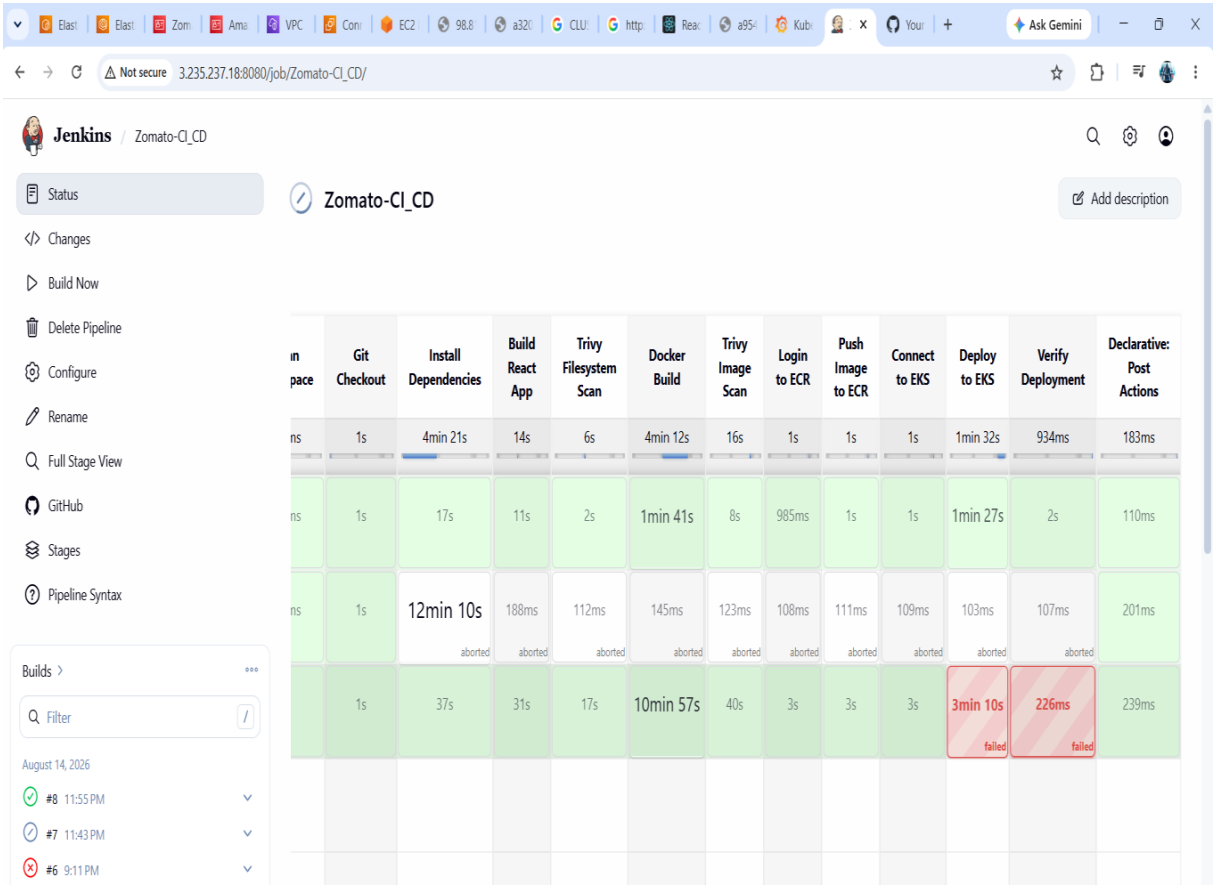


Figure 5.1 — Jenkins stage view across multiple pipeline runs, including one full green (successful) run.

Build #8 completed all twelve stages successfully. An earlier run, build #6, illustrates the pipeline's fail-fast behaviour: the Deploy to EKS and Verify Deployment stages failed (highlighted in red), so the release was stopped automatically before reaching production.

5.2 Build History

Pipeline	Zomato-CI_CD
Last Successful Build	#8 — 4 min 33 sec
Last Failed Build	#6 — 2 hr 48 min ago
Last Duration	4 min 0 sec

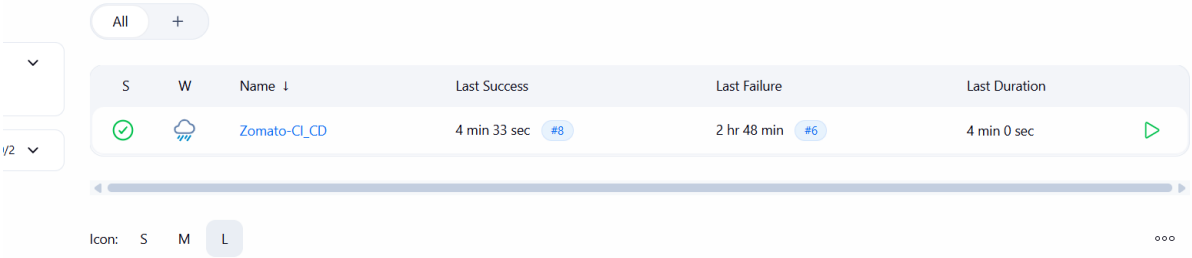


Figure 5.2 — Jenkins build-history summary for the Zomato-CI_CD pipeline.

6. Kubernetes Deployment & Verification

6.1 Confirming Pod Health

After the pipeline deploys the application, kubectl is used to confirm that both the application pod and the full monitoring stack are Running and Ready inside the EKS cluster.

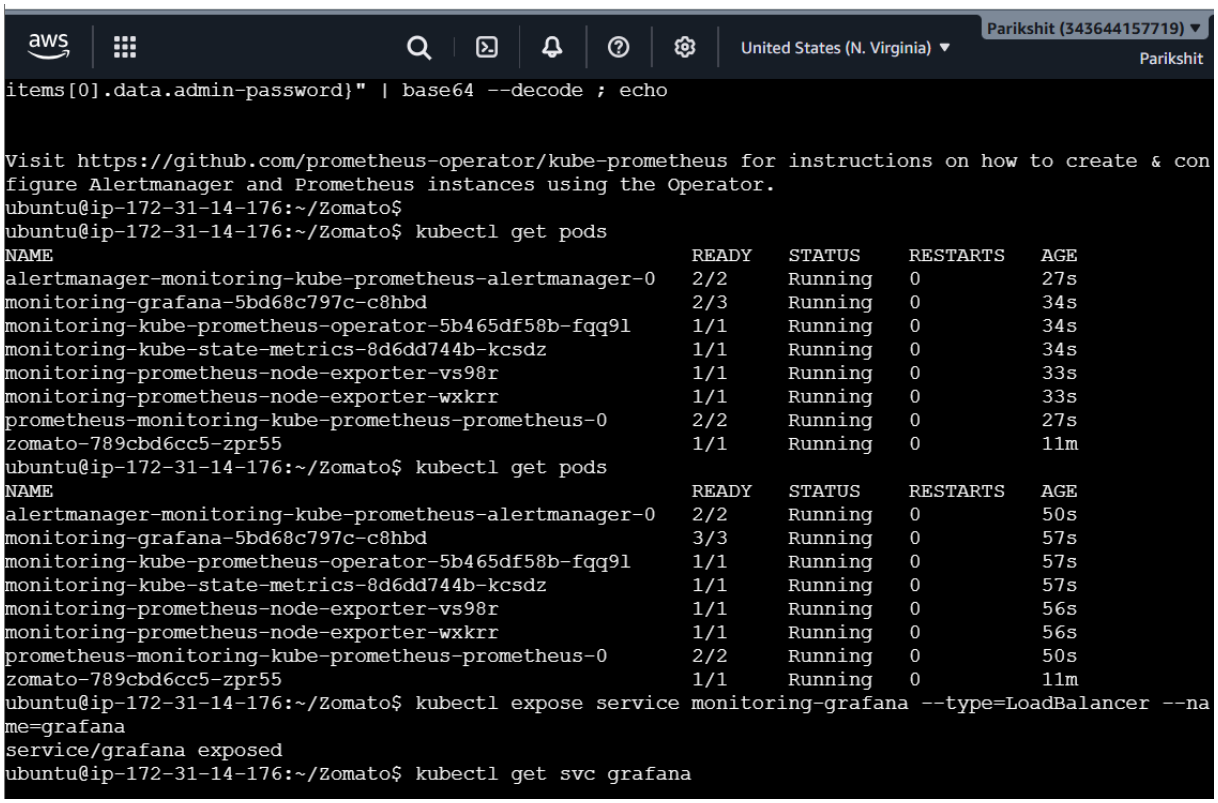


Figure 6.1 — kubectl get pods showing the zomato application pod and monitoring stack, all Running; the Grafana service is exposed via a LoadBalancer.

6.2 Application Live on AWS

The application is exposed to the internet through a Kubernetes LoadBalancer service, provisioning an AWS Elastic Load Balancer. Browsing the ELB's public DNS name confirms the food-ordering application is live and serving traffic from the EKS cluster.

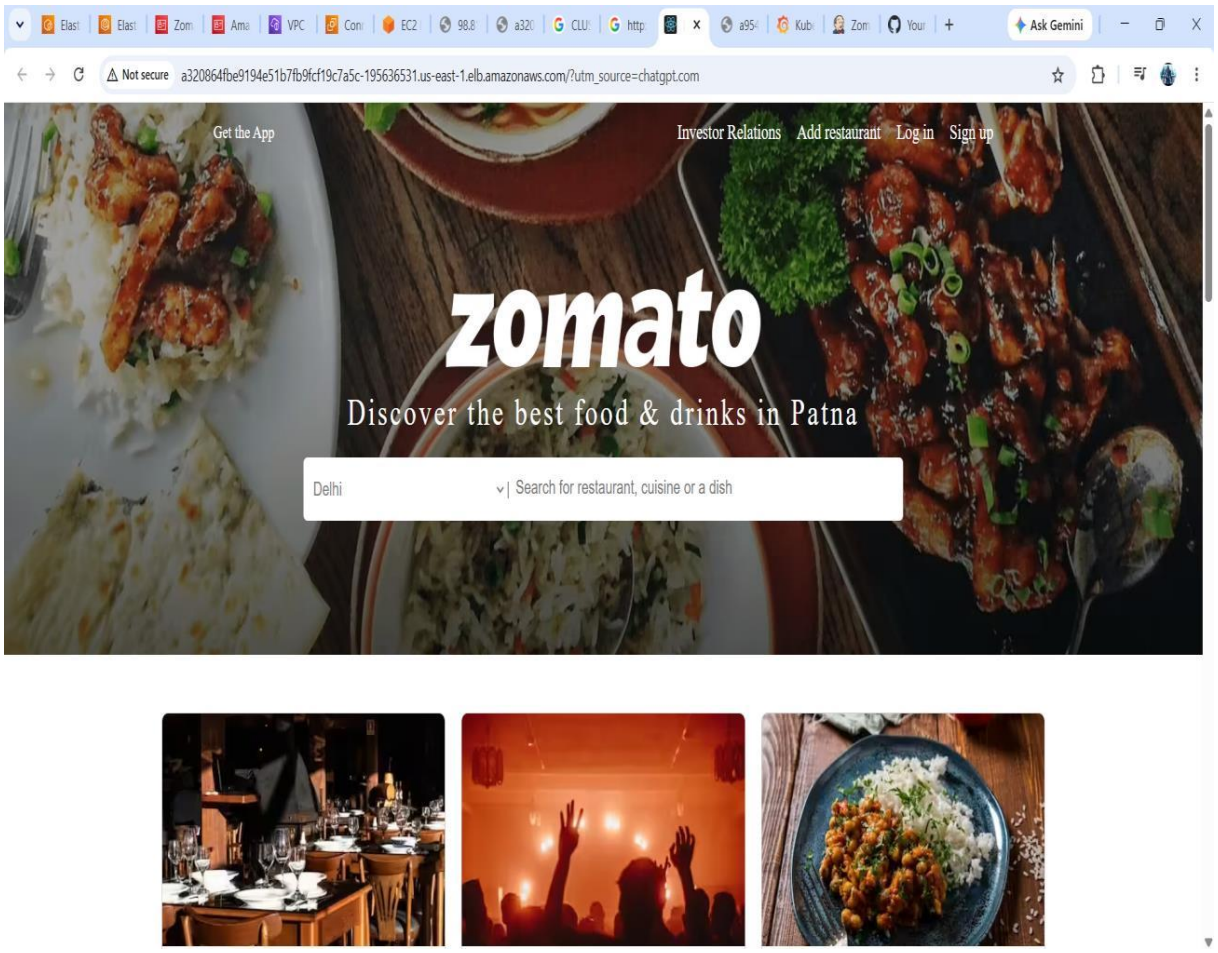


Figure 6.2 — The deployed Zomato clone, served through the AWS Elastic Load Balancer endpoint.

7. Monitoring & Observability

The kube-prometheus-operator stack (Prometheus, Alertmanager, kube-state-metrics, node-exporter, and Grafana) was deployed into the cluster to give real-time visibility into cluster health and resource consumption.

7.1 Cluster Resource Overview

Nodes	2
Namespaces	4
Global CPU Usage	Real 3.42% | Requested 18.0% | Limits 12.5%
Global RAM Usage	Real 58.58% | Requested 20.95% | Limits 40.32%
Total CPU (cores)	Real 0.137 | Requested 0.720 | Limits 0.500 | Total 4
Total RAM	Real 2.19 GiB | Requested 800 MiB | Limits 1.50 GiB | Total 3.73 GiB

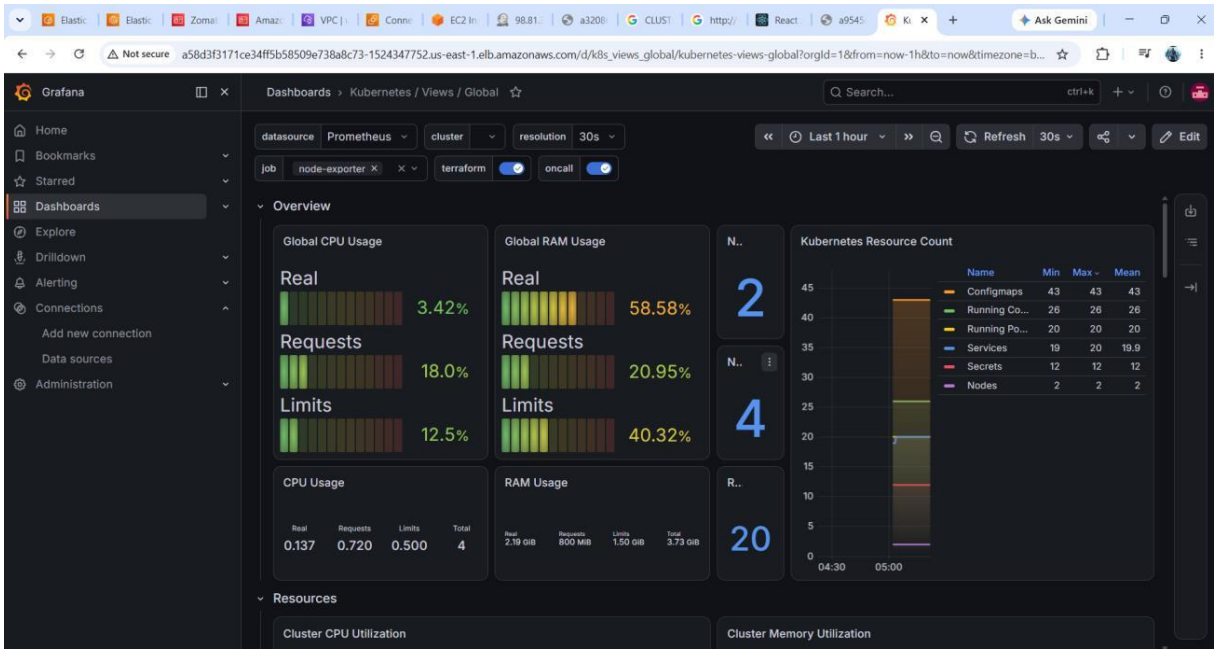


Figure 7.1 — Grafana "Kubernetes / Views / Global" dashboard showing cluster-wide CPU and RAM usage.

7.2 Per-Instance CPU & Memory Utilization

Drilling into the two worker-node instances shows utilization trending upward as the application and monitoring pods start, then settling into steady state — confirming the cluster has comfortable headroom under the current load.

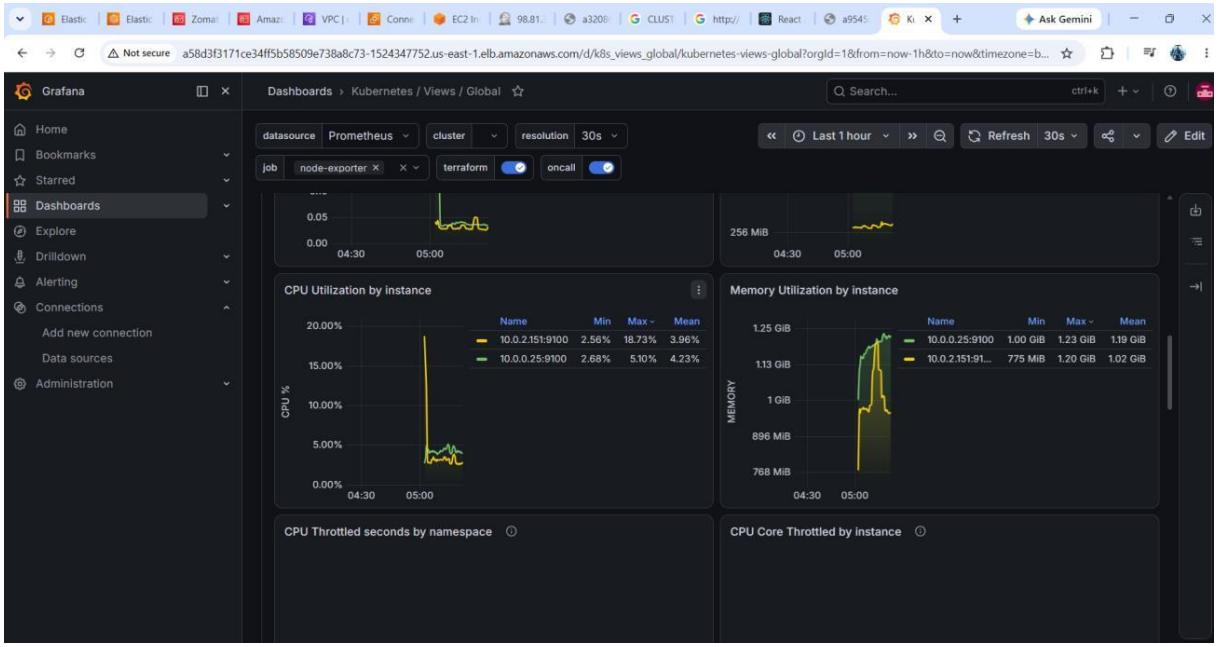


Figure 7.2 — CPU and memory utilization by worker-node instance over the last hour.

8. Security & Identity Access Management (IAM)

Following the principle of least privilege, the EKS worker nodes assume a dedicated IAM role rather than using long-lived credentials, and are granted only the managed policies required to operate as cluster nodes and pull images from ECR.

8.1 EKS Node IAM Role

Role Name	Zomato-EKS_NODe-Role
Purpose	Allows EC2 (worker node) instances to call AWS services on the cluster's behalf
Attached Policies	3 AWS-managed policies, incl. AmazonEC2ContainerRegistryPullOnly
Max Session Duration	1 hour

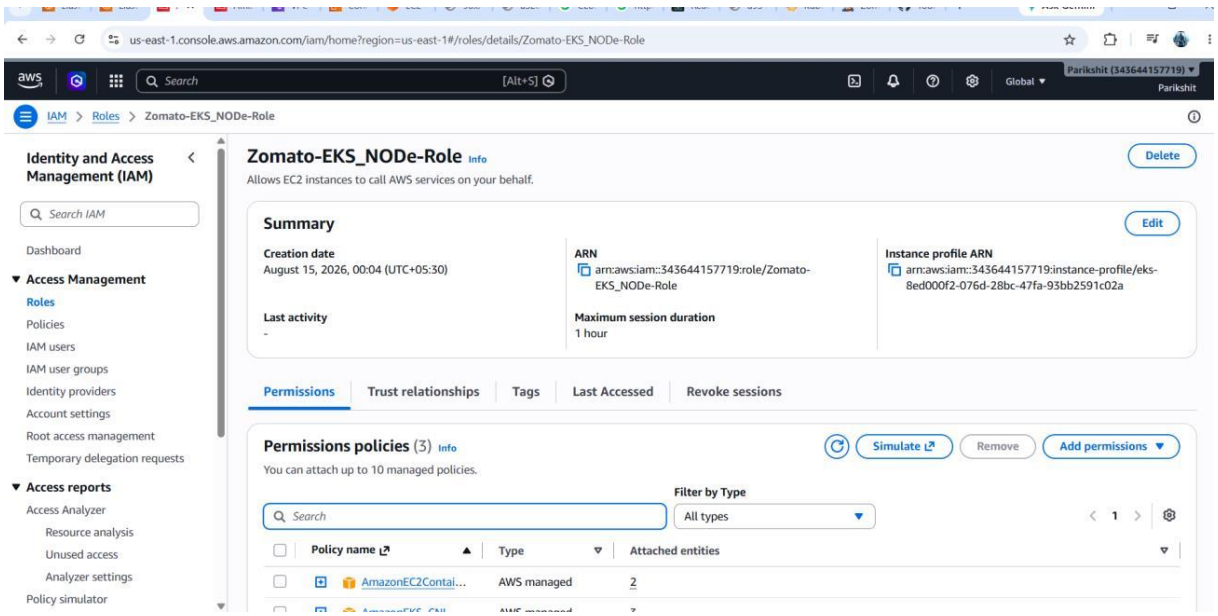


Figure 8.1 — Zomato-EKS_NODe-Role summary and attached permission policies.

8.2 ECR Pull Policy

The AmazonEC2ContainerRegistryPullOnly managed policy grants worker nodes read-only access to pull container images from ECR — nothing more — reducing the blast radius if a node were ever compromised.

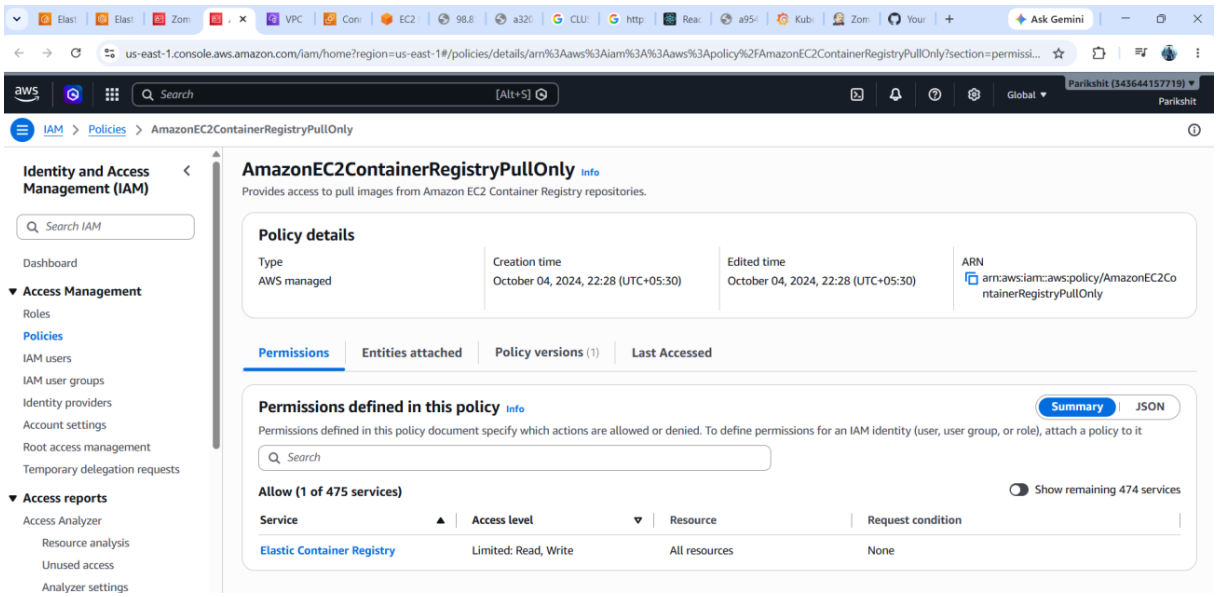


Figure 8.2 — AmazonEC2ContainerRegistryPullOnly policy: limited read access to Elastic Container Registry.

9. Source Control

Application code and pipeline configuration are version-controlled in GitHub, giving the Jenkins pipeline a single, auditable source of truth for every deployment.

Repository	Zomato
Owner	ParikshitArer
Visibility	Public
Primary Language	JavaScript
Origin	Forked and customized for this deployment

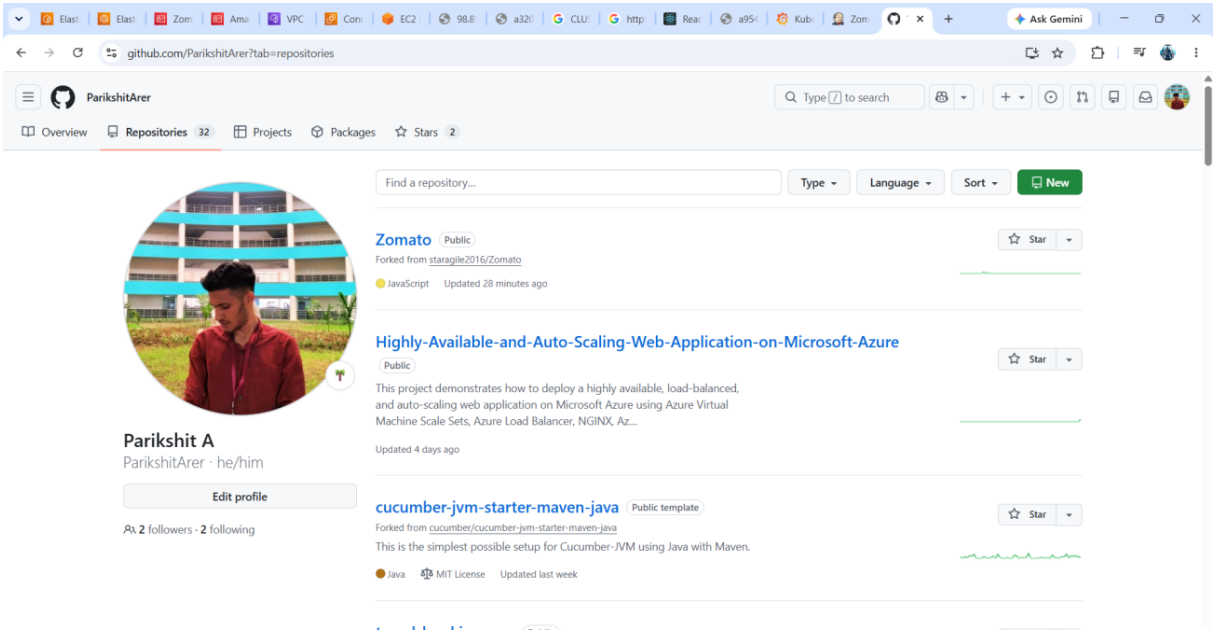


Figure 9.1 — GitHub repository hosting the Zomato application source code.

10. Summary & Outcomes

This implementation delivers a complete, automated path from source code to a monitored, highly-available deployment on AWS:

- A dedicated multi-AZ VPC gives the workload network isolation and fault tolerance.
- Amazon EKS (Kubernetes 1.36) with an autoscaling managed node group provides the compute platform.
- The application is containerized with Docker and versioned in a private Amazon ECR repository.
- A 12-stage Jenkins pipeline automates build, security scanning (Trivy), and deployment — failing fast on errors, as demonstrated by build #6, and completing cleanly on build #8.
- The application is verified to be Running in the cluster and reachable by end users through an AWS Load Balancer.
- Prometheus and Grafana give operators real-time visibility into cluster and node resource usage.
- IAM roles and AWS-managed, least-privilege policies govern what the cluster's nodes are allowed to do.

Together these components move Zomato's release process from manual and monolithic to automated, containerized, and observable — meeting the project's goals of high availability, faster releases, and automated scalability.